ELK - Extended Logic Keys

Slot reference: every hotkey slot, tab by tab in the order of the in-game window: ELK, Squad (stock), AtmosphereAutopilot (and its Approach Autopilot add-on), MechJeb2, MechJeb2+, NavUtilities

ELK version **1.1.4** for KSP 1.12.5. Source, releases and README: https://github.com/Rjoande/ELK. License MIT. Document date 2026-11-08.

Supported mods and the versions this document was verified against

AtmosphereAutopilot 1.6.1 by Boris-Barboris: https://github.com/Boris-Barboris/AtmosphereAutopilot.
Approach Autopilot 1.0.1 (formerly **AtmosphereApproach**, 1.0.0) by Rjoande: https://github.com/Rjoande/ApproachAutopilot. An add-on of AtmosphereAutopilot, so its key sits in the AA tab. ELK recognises either name; with both installed the key goes to AtmosphereApproach, since Approach Autopilot disables itself until the old folder is deleted.
MechJeb2 2.15.3 by the MuMech team: https://github.com/MuMech/MechJeb2.
NavInstruments Continued (NavUtilities) 0.8.1.1 by linuxgurugamer: https://github.com/linuxgurugamer/NavInstruments.

All mods are reached by reflection only: nothing bundled, nothing referenced. A mod that is not installed hides its tab and turns its slots into no-ops. Every key calls the member the stock UI, the mod's own window or its native hotkey calls. All keys ship empty. "Slot id" is the node name in the cfg file. Map view: a key fires there only if its row's **Map** button is on (default on for the SAS keys only; the key-set keys always fire).

ELK - 2 slots

Tab **ELK**, file *ELK.cfg*. Options: **enabled** (true): mod master switch, off = no ELK key fires anywhere. **sets**: comma-separated list of key-set names, in rotation order (empty = sets off, every key always active); each key row of the other tabs then gets a set button (empty = always active). **active_set**: default set. **sets_per_vessel** (true): each vessel remembers its own set in the save. **sets_disengage** (all sets): leaving one of these sets in flight switches off what its keys control (SAS off, brakes released, AA master off, SmartASS / Translatron off, landing or node execution aborted); one checkbox per set in the tab.

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight
Key set: next	`SET_NEXT`	`ElkSets.Step(+1)`	Activates the next key set in the list (wraps around) and announces it on screen. No set declared: nothing. Always fires, map view included.
Key set: previous	`SET_PREV`	`ElkSets.Step(-1)`	Activates the previous key set. Same rules.

Squad (stock) - 15 slots

Tab **Squad**, file *ELK_Squad.cfg*. Options: **sas_autoengage** (true): a SAS mode key pressed with SAS off switches SAS on and selects the mode; off = the key is a no-op while SAS is off. **sas_navball** (true): a SAS mode key that actually selects a mode also reopens the navball if it is hidden. **gear_resync**, **lights_resync** (false): see Gear and lights. A mode the vessel cannot hold (pilot level, probe core, no target or node) is refused before SAS is touched, as on the navball.

Brakes

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight
Brakes: toggle	`BRAKES_TOGGLE`	`ActionGroups.ToggleGroup(Brakes)`	Latches the brakes on, like a click on the BRAKES button of the flight UI; pressed again, releases them. The stock Brakes key (B) holds the brakes only while held and releases them on key-up, so do not bind this slot to B, with or without modifiers: it could only release.

No BRAKES_ON / BRAKES_OFF on purpose: stock B already sets the brakes on while pressed and off on release, so "off" is a tap of B and an "on" key would be the toggle without the release.

Gear and lights (no switch-off when leaving a key set)

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight
Gear: down	`GEAR_DOWN`	`ActionGroups.SetGroup(Gear, true)`	Gear down, like a click on the GEAR button of the flight UI when it is off (stock G only toggles). GEAR already on: nothing, even if a leg was retracted from its right-click menu (that menu never updates the GEAR light). With gear_resync on, the Deploy command is sent again to every Gear action: legs already down stay down.
Gear: up	`GEAR_UP`	`ActionGroups.SetGroup(Gear, false)`	Gear up, the same way. With gear_resync on and GEAR already off, the Retract command is sent again.
Lights: on	`LIGHTS_ON`	`ActionGroups.SetGroup(Light, true)`	Lights on, like a click on the LIGHTS button when it is off (stock U only toggles). LIGHTS already on: nothing, unless lights_resync is on, which sends the On command again (a lamp switched off from its menu comes back; a blinking lamp restarts its blink). On EVA also the helmet lamp (KerbalEVA.ToggleLamp).

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight
Lights: off	LIGHTS_OFF	ActionGroups.SetGroup(Light, false)	Lights off, the same way, lights_resync included.

Resync sends the command to every action in the group: gear, lights and any action that follows on/off stay in line, but a part whose action does not (a one-way Extend/Retract, some toggles) flips on every resend. With gear_resync or lights_resync on, keep such parts out of the Gear / Lights groups.

SAS modes (sas_autoengage, sas_navball apply)

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight
SAS: Stability Assist	SAS_STABILITY	CanSetMode; SetGroup(SAS, true) if off; Autopilot.Enable(StabilityAssist)	Holds the current attitude.
SAS: Prograde	SAS_PROGRADE	CanSetMode; SetGroup(SAS, true) if off; Autopilot.Enable(Prograde)	Points prograde (navball marker, current speed mode).
SAS: Retrograde	SAS_RETROGRADE	CanSetMode; SetGroup(SAS, true) if off; Autopilot.Enable(Retrograde)	Points retrograde.
SAS: Normal	SAS_NORMAL	CanSetMode; SetGroup(SAS, true) if off; Autopilot.Enable(Normal)	Points normal (navball marker).
SAS: Anti-normal	SAS_ANTINORMAL	CanSetMode; SetGroup(SAS, true) if off; Autopilot.Enable(Antinormal)	Points anti-normal.
SAS: Radial in	SAS_RADIAL_IN	CanSetMode; SetGroup(SAS, true) if off; Autopilot.Enable(RadialOut)	Points radial in, towards the body (navball marker). Stock's enum names are inverted, hence RadialOut.
SAS: Radial out	SAS_RADIAL_OUT	CanSetMode; SetGroup(SAS, true) if off; Autopilot.Enable(RadialIn)	Points radial out, away from the body. Same inversion.
SAS: Target	SAS_TARGET	CanSetMode; SetGroup(SAS, true) if off; Autopilot.Enable(Target)	Points at the selected target. No target: refused.
SAS: Anti-target	SAS_ANTITARGET	CanSetMode; SetGroup(SAS, true) if off; Autopilot.Enable(AntiTarget)	Points away from the selected target. No target: refused.
SAS: Maneuver node	SAS_MANEUVER	CanSetMode; SetGroup(SAS, true) if off; Autopilot.Enable(Maneuver)	Points along the next maneuver node. No node: refused.

AtmosphereAutopilot - 29 slots

Tab **AtmosphereAutopilot**, file *ELK_AtmosphereAutopilot.cfg*. Options: **aa_autoengage** (true): a Cruise Flight or FBW key pressed while that controller is off switches master and controller on first; off = the key is a no-op until the controller is active. **aa_autoshow** (false): open the AA window when the master is switched on. **aa_vs_step** (1), **aa_alt_step** (50), **aa_hdg_step** (1), **aa_spd_step** (5): bug increments. Column "AA native key": the AA hotkey with the same effect ("-" = AA has none, the action exists only as an ELK key); the tab's Import / Export / Restore buttons move bindings between ELK and AA's *Global_settings.txt*.

Manager

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight	AA native key
AA: Master switch	AA_MASTER_TOGGLE	TopModuleManager.Active (setter) + mainMenuGUIUpdate()	Master on/off, like AA's native master key.	master_switch_key
AA: FD (Cruise <-> FBW)	AA_FD_TOGGLE	TopModuleManager.activateAutopilot(CruiseController | StandardFlyByWire)	With AA on: from Cruise Flight to Standard FBW, from anything else to Cruise Flight. With AA off: switches it on with Standard FBW.	-
AA: Show/hide window	AA_GUI_TOGGLE	GUIWindow.ToggleGUI() on the manager	Shows or hides AA's main window, like the native Shift+master key.	-
AA: Standard FBW	AA_SELECT_FBW	activateAutopilot(StandardFlyByWire)	Makes Standard Fly-By-Wire the active controller (master on if needed).	-
AA: Cruise Flight	AA_SELECT_CRUISE	activateAutopilot(CruiseController)	Makes Cruise Flight the active controller (master on if needed).	-

Cruise Flight (aa_autoengage applies)

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight	AA native key
CF: Level (LVL)	AA_CF_LEVEL	CruiseController.LevelFlightMode = true	Lateral mode Level: wings level, holds the current heading.	-

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight	AA native key
CF: Heading (HDG)	AA_CF_HEADING	CruiseController.CourseHoldMode = true	Lateral mode Heading: flies the HDG bug. AA itself refuses heading hold beyond 80 degrees of latitude.	-
CF: Waypoint (NAV)	AA_CF_WAYPOINT	CruiseController.WaypointMode = true	Lateral mode Waypoint: navigates to the destination set in the Cruise Flight window.	-
CF: V/S engage/disengage	AA_CF_VS_TOGGLE	vertical_control = true; height_mode = VerticalSpeed	Vertical control in V/S mode; pressed again in that mode, disengages it.	-
CF: ALT engage/disengage	AA_CF_ALT_TOGGLE	vertical_control = true; height_mode = Altitude	Altitude hold; pressed again in that mode, disengages it.	-
CF: Vertical motion on/off	AA_CF_VERTICAL_TOGGLE	CruiseController.vertical_control	Vertical control on/off without changing its mode.	vertical_control_key
CF: ALT <-> V/S-FPA	AA_CF_SETPOINT_TYPE_TOGGLE	CruiseController.height_mode (+ prev_height_change_mode_by_hotkey)	Altitude hold to the last rate mode used (V/S or FPA) and back, like AA's native key.	toggle_vertical_setpoint_type_key
CF: Keys input mode	AA_CF_KEYS_MODE_TOGGLE	CruiseController.use_keys	Move the bugs with the stock pitch and yaw keys, on/off.	switch_key_mode
CF: V/S bug up	AA_CF_VS_UP	desired_vertsetpoint.Value += aa_vs_step	Vertical setpoint up one step (m/s in V/S mode, degrees in FPA mode).	-
CF: V/S bug down	AA_CF_VS_DOWN	desired_vertsetpoint.Value -= aa_vs_step	Lowers the vertical setpoint by one step.	-
CF: ALT bug up	AA_CF_ALT_UP	desired_altitude.Value += aa_alt_step	Raises the target altitude (ASL) by one step, in metres. Moves the bug only.	-
CF: ALT bug down	AA_CF_ALT_DOWN	desired_altitude.Value -= aa_alt_step	Lowers the target altitude by one step.	-
CF: ALT bug = current	AA_CF_ALT_SYNC	desired_altitude.Value = vessel.altitude; vertical_control = true; height_mode = Altitude	ALT bug = current ASL altitude, altitude hold engaged (never disengaged).	-
CF: HDG bug up	AA_CF_HDG_UP	desired_course.Value += aa_hdg_step (wrapped 0-360)	Turns the target heading right by one step, in degrees. Moves the bug only.	-
CF: HDG bug down	AA_CF_HDG_DOWN	desired_course.Value -= aa_hdg_step (wrapped 0-360)	Turns the target heading left by one step.	-
CF: HDG bug = current	AA_CF_HDG_SYNC	desired_course.Value = FlightGlobals.ship_heading; CourseHoldMode = true	Sets the HDG bug to the current compass heading and selects heading hold.	-

Standard Fly-By-Wire (aa_autoengage applies)

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight	AA native key
FBW: Moderation	AA_FBW_MODERATION	StandardFlyByWire.moderation_switch	Toggles angle-of-attack and G-load moderation, like AA's native moderation key.	moderation_keycode
FBW: Rocket mode	AA_FBW_ROCKET	StandardFlyByWire.RocketMode	Toggles FBW rocket mode.	rocket_mode_keycode
FBW: Coordinated turn	AA_FBW_COORD_TURN	StandardFlyByWire.Coord_turn	Toggles coordinated turns.	coord_turn_keycode

Speed and thrust (no auto-engage)

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight	AA native key
AA: Speed control	AA_SPEED_CONTROL	Prograde ThrustController.spd_control_enabled	Automatic speed (throttle) control on/off; effective only while a controller using it is active.	spd_control_toggle_key
AA: SPD bug up	AA_SPD_UP	Prograde ThrustController.setpoint.value + setpoint_field.Value += aa_spd_step	Raises the speed setpoint by one step in the unit chosen in AA's window (m/s, kts, IAS, kIAS); Mach by step x 0.01. Also with speed control off (preset); never switches anything on.	-
AA: SPD bug down	AA_SPD_DOWN	Prograde ThrustController.setpoint.value + setpoint_field.Value -= aa_spd_step	Lowers the speed setpoint by one step, never below 0.	-
AA: Thrust balancing	AA_THRUST_BALANCING	FlightModel.balance_engines	Toggles automatic thrust balancing between engines.	balancing_toggle_key

Approach Autopilot / AtmosphereApproach add-on (no auto-engage)

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight	AA native key
APR: Arm/disarm	`AA_APR_TOGGLE`	`ApproachController.Armed (setter)`	Arms or disarms APR on the runway tuned in NavInstruments, like the add-on's own hotkey: it only flips the arm state, without selecting the controller or touching the master, so whatever is flying keeps flying until localizer capture. No runway tuned: the add-on refuses. Controller not created yet (AA master never switched on for this vessel): nothing. Row shown only with the add-on installed.	`apr_arm_key`

MechJeb2 - 27 slots on two tabs

Tabs **MechJeb2** and **MechJeb2+**, one file *ELK_MechJeb2.cfg*. Every key acts on the active vessel's master MechJeb on each press (no MechJeb aboard: no-op); a module locked in career is never driven. Options: **mj_autoengage** (true): a Translatron speed or Kill H/S key pressed with the Translatron off switches it on in Keep vertical first. **mj_autoshow** (false): also open the window of the module driven. **mj_trans_step** (1 m/s): speed key increment. **Copy from / to SAS**: copy the ten stock SAS bindings onto the matching SmartASS slots, or back (mapping below). The Translatron is a throttle controller with one mode at a time; the speed keys write its Speed box and live setpoint together, like the window's + / 0 / - buttons.

Tab MechJeb2: MechJeb's own part actions (Speed = 0 and Kill H/S driven directly)

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight
SmartASS: Off	`MJ_SASS_OFF`	`MechJebCore.OnDeactivateSmartASSAction (SmartASS target OFF)`	Disengages SmartASS attitude control.
SmartASS: Kill rotation	`MJ_SASS_KILLROT`	`OnKillRotationAction (Target.KILLROT)`	Cancels rotation and holds the current attitude.
SmartASS: Prograde	`MJ_SASS_PROGRADE`	`OnOrbitProgradeAction (Mode.ORBITAL, Target.PROGRADE)`	Points orbital prograde.
SmartASS: Retrograde	`MJ_SASS_RETROGRADE`	`OnOrbitRetrogradeAction (Target.RETROGRADE)`	Points orbital retrograde.
SmartASS: Normal+	`MJ_SASS_NORMAL_PLUS`	`OnOrbitNormalAction (Target.NORMAL_PLUS)`	Points orbit normal (+).
SmartASS: Normal-	`MJ_SASS_NORMAL_MINUS`	`OnOrbitAntinormalAction (Target.NORMAL_MINUS)`	Points orbit anti-normal (-).
SmartASS: Radial+	`MJ_SASS_RADIAL_PLUS`	`OnOrbitRadialOutAction (Target.RADIAL_PLUS)`	Points radial out, away from the body (navball radial-out marker).
SmartASS: Radial-	`MJ_SASS_RADIAL_MINUS`	`OnOrbitRadialInAction (Target.RADIAL_MINUS)`	Points radial in, towards the body.
Translatron: Off	`MJ_TRANS_OFF`	`OnTranslatronOffAction (Translatron.SetMode(OFF))`	Switches the Translatron off.
Translatron: Keep vertical	`MJ_TRANS_KEEP_VERT`	`OnTranslatronKeepVertAction (SetMode(KEEP_VERTICAL))`	Holds the vertical speed in the Speed box (0 = hover, negative = descent). Throttle only: attitude is yours unless Kill H/S is on.
Translatron: Speed = 0	`MJ_TRANS_ZERO`	`Translatron.trans_spd.Val = 0; Thrust.TransSpdAct = 0`	The window's "0" button: Speed box and setpoint to 0 in the current mode (hover in Keep vertical). Translatron off: Keep vertical first with mj_autoengage, otherwise no-op.
Translatron: Kill H/S	`MJ_TRANS_KILLH_TOGGLE`	`Thrust.TransKillH (+ Translatron.SetMode(KEEP_VERTICAL))`	Kill horizontal speed: in Keep vertical the Translatron also steers against the horizontal velocity, cancelling drift. Translatron on: toggles the flag. Off: with mj_autoengage engages Keep vertical with the flag on, otherwise toggles the flag only.
Translatron: PANIC	`MJ_PANIC`	`OnPanicAction (Translatron.PanicSwitch())`	Launch abort: throttle off, jettisons everything below the last engine stage and fires it (escape motor), 2 s full-throttle climb, then the landing autopilot lands somewhere. Pressed again: cancels.
Landing: Land somewhere	`MJ_LAND_SOMEWHERE`	`OnLandsomewhereAction (Landing.LandUntargeted)`	Starts the landing autopilot with no target.
Landing: Land at KSC	`MJ_LAND_KSC`	`OnLandTargetAction (LandingGuidance.SetAndLandTargetKSC)`	Targets the KSC pad and starts the landing autopilot on it.
Ascent: Autopilot on/off	`MJ_ASCENT_TOGGLE`	`OnAscentAPToggleAction (Ascent.Users add/remove)`	Engages or disengages the ascent autopilot (Ascent Guidance settings).

Tab MechJeb2+: what the part actions do not cover

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight
SmartASS: Node	`MJ_SASS_NODE`	`SmartASS.target = NODE; Engage()`	Points along the next maneuver node. No planned node: nothing happens.
SmartASS: Target+	`MJ_SASS_TARGET_PLUS`	`SmartASS.mode = TARGET; target = TARGET_PLUS; Engage()`	Points at the selected target. No target: nothing happens.
SmartASS: Target-	`MJ_SASS_TARGET_MINUS`	`SmartASS.mode = TARGET; target = TARGET_MINUS; Engage()`	Points away from the selected target. No target: nothing happens.
Translatron: Keep surface	`MJ_TRANS_KEEP_SURF`	`Translatron.SetMode(KEEP_SURFACE)`	Holds the surface speed in the Speed box by throttle alone, pointing prograde or retrograde; never turns the vessel. Kill H/S has no effect here.
Translatron: Keep orbital	`MJ_TRANS_KEEP_ORBIT`	`Translatron.SetMode(KEEP_ORBITAL)`	Same for orbital speed.

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight
Translatron: Speed +step	MJ_TRANS_SPEED_UP	trans_spd.Val += mj_trans_step; TransSpdAct = trans_spd	The window's "+" button: Speed box and setpoint up one step (m/s). Translatron off: Keep vertical first with mj_autoengage, otherwise no-op.
Translatron: Speed -step	MJ_TRANS_SPEED_DOWN	trans_spd.Val -= mj_trans_step; TransSpdAct = trans_spd	The window's "-" button: one step down. Same rules.
Landing: Land at target	MJ_LAND_TARGET	Landing.LandAtPositionTarget(LandingGuidance)	Lands on the position target set in Landing Guidance. Needs a position target and a vessel not landed.
Landing: Abort	MJ_LAND_ABORT	Landing.StopLanding()	Aborts the landing autopilot, only while it is running.
Node: Execute / abort	MJ_NODE_EXECUTE_TOGGLE	Node.ExecuteOneNode(ManeuverPlanner) / Node.Abort()	Executes the next maneuver node; pressed while running, aborts it. No node or zero delta-v: nothing.
MechJeb: Show/hide menu	MJ_MENU_TOGGLE	MechJebModuleMenu.ShowHideWindow()	Shows or hides MechJeb's main menu window.

Copy from / to SAS mapping

Stock SAS slot	SmartASS slot
SAS_STABILITY	MJ_SASS_KILLROT
SAS_PROGRADE	MJ_SASS_PROGRADE
SAS_RETROGRADE	MJ_SASS_RETROGRADE
SAS_NORMAL	MJ_SASS_NORMAL_PLUS
SAS_ANTINORMAL	MJ_SASS_NORMAL_MINUS

Stock SAS slot	SmartASS slot
SAS_RADIAL_IN	MJ_SASS_RADIAL_MINUS
SAS_RADIAL_OUT	MJ_SASS_RADIAL_PLUS
SAS_TARGET	MJ_SASS_TARGET_PLUS
SAS_ANTITARGET	MJ_SASS_TARGET_MINUS
SAS_MANEUVER	MJ_SASS_NODE

NavUtilities (NavInstruments Continued) - 1 slot

Tab **NavUtilities**, file *ELK_NavUtilities.cfg*, no options.

ELK toolbar label	Slot id (cfg node)	Member driven	What the key does in flight
NavUtilities: HSI window	NAV_HSI_TOGGLE	NavUtilLibApp.displayHSI() + toolbar icon sync (SetTrue/SetFalse)	Opens or closes the HSI window, like a left click on the NavUtilities toolbar button (icon kept in sync).